

Resonance

ALS Voice, Motor and Communication Companion

Complete build plan and technical specification for an iOS app (Swift, SwiftUI, Xcode)
2026 Congressional App Challenge | California's 10th District

Submission deadline
Monday, October 26, 2026 at 12:00 p.m. Eastern = 9:00 a.m. Pacific

Plan date
Friday, October 23, 2026 | Version 1.0 | Working title: rename freely

Resonance is a monitoring and communication support tool. It does not diagnose ALS or any other condition.

Contents

#	Section
1	How to use this document, building it with Claude, and the accounts and tools you need
2	Executive summary and why this can win CA-10
3	Background: ALS, speech, and what is measurable
4	Product overview: modules, users and a week of use
5	Screens and navigation
6	Architecture
7	Xcode project setup
8	Data model (SwiftData)
9	Module spec: Voice Lab (audio analysis)
10	Module spec: Motor Lab (motor assessment)
11	Module spec: Daily Function Check (patient-reported data)
12	Module spec: Insights Engine and AI Speech Change Score
13	Module spec: Talk board (hands-free communication)
14	Module spec: Voice Bank (Personal Voice and message banking)
15	Module spec: Care Report, onboarding, demo mode, settings
16	Build tasks T01 to T24 with acceptance criteria
17	Four-day schedule and what to cut if behind
18	Testing plan
19	Demo video script (3 minutes)
20	Submission checklist and AI-use disclosure
21	Risks, ethics and privacy
22	Prompts to give an AI coding assistant
23	Sources

1. How to use this document

This is a complete specification for **Resonance**, an iPhone app for people living with ALS. It is written so you can build it yourself in Xcode, or hand sections to an AI coding assistant one task at a time. Every module lists what the user sees, what the app measures, the exact Apple frameworks to use, the formulas, and a checklist that defines when the work is done.

Rules for an AI coding assistant working from this plan

1. Build the tasks in Section 16 **in order**. Do one task per session. Read the module spec the task points to before writing code.
2. Target **iOS 17.0+**, **Swift 5.9 or later**, **SwiftUI** for all UI, **SwiftData** for storage, and the **@Observable** macro for view models. No UIKit storyboards.
3. **No network calls and no third-party packages.** Everything uses Apple frameworks: AVFoundation, Speech, Accelerate, CoreMotion, ARKit, Charts, SwiftData, CoreML. All data stays on the device.
4. Keep algorithms in plain Swift structs with no UI imports, so they can be unit-tested (Section 18). Put each type in the file named in Section 7.
5. Never present any output as a diagnosis. Use the wording in Section 21 for every score and alert.
6. When a task's "Done when" checklist passes, stop and summarize what changed, which files were touched, and anything not finished.
7. If an Apple API in this plan has changed or is unavailable, use the closest current equivalent and note the change in a code comment.

How you (the student) should use it

- Read Sections 2 to 5 tonight so you understand the whole app before writing code.
- Follow the schedule in Section 17. It assumes about 26 hours of work between Friday evening and Sunday night.
- Keep a running notes file of what you built and which parts AI helped with; you need it for the AI-use disclosure (Section 20).
- You need a Mac with Xcode, and an iPhone with Face ID (TrueDepth camera) for the hands-free features. A free Apple ID is enough to run the app on your own phone.

Can Claude build this app for you?

Claude can write nearly all of the code in this plan. But an iPhone app must be compiled with **Xcode on a Mac** and tested on a **real iPhone**, so what Claude can do depends on where it runs:

Where Claude runs	What it can do	What it cannot do
A cloud chat session not connected to your Mac	Write every Swift file, test, README and CLAUDE.md, and send them to you as files	Compile the app, open Xcode, or run anything on your iPhone
Claude desktop app with your project folder connected	Write files straight into your Xcode project; if it has a shell on your Mac, run xcodebuild to compile and fix build errors	Tap through the app on your phone, grant permissions, or judge whether a measurement looks right
Claude Code in your Mac's Terminal, opened in the project folder (recommended)	Everything above, plus run the unit tests (xcodebuild test) and train the Create ML model with a Swift script	The same physical-testing limits

Set your expectations

No AI tool can promise a fully working app from one prompt. The realistic loop for every task: Claude writes the code and gets it to compile; you press Run in Xcode to install it on your iPhone and test the "Done when" list; you tell Claude exactly what happened; Claude fixes it.

The microphone, motion sensors and face tracking only work on a real phone, so only you can check those. The schedule in Section 17 already includes this loop.

Accounts, tools and keys checklist

Item	Needed?	Cost	Notes
Mac with the latest Xcode (Mac App Store)	Required	Free	Install tonight; Xcode is a very large download.
Apple ID signed into Xcode (Personal Team)	Required	Free	Lets you install on your own iPhone. Apps installed this way stop opening after 7 days; just press Run again. The paid Apple Developer Program is not needed.
iPhone with Face ID, iOS 17 or later	Required	-	Needed for head-pointer control. Turn on Developer Mode (Settings -> Privacy and Security) after the first install attempt.
API keys (OpenAI, Google, etc.)	**Not needed**	-	Every feature uses Apple's on-device frameworks. Nothing is sent to a server.
GitHub account	Required	Free	The submission needs a link to your code.
Congressional App Challenge registration	Required	Free	congressionalappchallenge.us, California's 10th District.
YouTube or Vimeo account	Required	Free	Upload the demo video as unlisted.
Synapse account	Only for the AI stretch task	Free	To download the VOC-ALS voice dataset (Section 12).
Claude desktop app or Claude Code	Recommended	Your plan	To write and compile code directly in your project folder.

Recommended workflow with Claude

1. Install Xcode and create the empty project exactly as in Section 7. Those first clicks, signing in, and the first run on your iPhone are things only you can do.
2. Put this PDF in the project folder as **PLAN.pdf** and the **CLAUDE.md** file that came with it in the folder root. Claude Code reads CLAUDE.md automatically at the start of every session.
3. Open Claude Code in the project folder (or connect the folder in the Claude desktop app) and paste the starting prompt from Section 22 with task T02.
4. After each task: Claude compiles with xcodebuild; you press Run in Xcode, test on your iPhone, and report exactly what happened.
5. Commit to GitHub when a task passes. Keep a list of what Claude wrote for the AI-use disclosure.

Key terms

ALS: amyotrophic lateral sclerosis, a progressive disease of the nerve cells that control muscles. **Bulbar**: affecting the muscles for speech and swallowing. **Dysarthria**: slurred or slow speech caused by muscle weakness. **DDK rate** (diadochokinetic rate): how fast a person can repeat syllables like "pa-ta-ka". **F0**: fundamental frequency, the pitch of the voice. **Baseline**: a person's own starting values; the app compares people to themselves, not to others. **Dwell selection**: choosing a button by pointing at it and holding still for a set time. **Message banking**: recording phrases in your own voice to play later.

2. Executive summary

The problem. ALS gradually weakens the muscles for speaking, moving and eventually breathing. Changes in speech and hand function are usually checked at clinic visits that can be months apart, so families often notice decline late. And many people with ALS lose the ability to speak, which cuts them off from the people they love.

The solution. Resonance is an on-device iPhone app that does two jobs every day:

- **Measures change.** Short weekly check-ins analyze the person's voice (audio analysis), test finger speed and steadiness (motor assessment), and record how daily activities are going (patient-reported data). An insights engine and an on-device AI speech model turn these into trends and a printable report for the care team.
- **Keeps the person's voice.** A hands-free Talk board lets the user speak by moving their head or eyes, using phrases recorded in their own voice or Apple's Personal Voice, so communication continues as speech and hand movement fade.

How it matches the Synapse Match formula

Synapse Match, the 2025 CA-10 winner from Emerald High School in Dublin, used existing patient data, an audio analysis tool and a motor assessment with AI to estimate the likelihood of Parkinson's disease. Resonance uses the same three ingredients plus AI, but points them at a different problem: **monitoring ALS progression and preserving communication** rather than detection. That keeps the technical depth judges rewarded last year while making clear it is not a copy.

Ingredient	Synapse Match (2025)	Resonance (2026)
Patient data	Existing patient data	Daily Function Check plus session history
Audio analysis	Audio analysis tool	Voice Lab: sustained vowel, syllable repetition, reading passage; 12 speech measures
Motor assessment	Motor assessment	Motor Lab: alternating finger tapping, steadiness hold, optional spiral trace
AI	Predicts likelihood of Parkinson's	Speech Change Score (Core ML) and trend detection against the person's own baseline
Purpose	Early identification	Weekly monitoring, care-team report, and hands-free communication in the user's own voice

Why it fits what CA-10 judges have picked

Year	CA-10 winner	Pattern Resonance shares
2025	Synapse Match: AI, audio and motor tests for early Parkinson's signs	Same three measurement types, neurological condition
2024	NeuroChess: AI analysis of chess games to build focus for ADHD	AI applied to a brain-based condition
2023	CogniPro: game plus eye tracking to reduce ADHD misdiagnosis	Eye tracking (Talk board)
2022	TrueDetect: wearable biosensors tracking stress and anxiety	Sensors producing a trend over time

Your one-sentence pitch

"Resonance measures how ALS is changing a person's voice and hands every week, shares that with their care team, and keeps them talking, in their own voice, hands-free, when speech fades."

3. Background: ALS, speech and what is measurable

Use this section for your video, README and the "why" in your submission. Keep claims general unless you cite a source.

- ALS affects motor neurons, so it weakens voluntary muscles over time. In **bulbar-onset** ALS, speaking and swallowing are affected first.
- Research has explored home-recorded **digital speech measures** for ALS. A small pilot study recorded people with ALS at home on a tablet twice a month, measuring speaking rate on a reading passage, DDK rate on repeated "pa", and pitch on a held vowel. People with normal speech had significantly faster speaking rates than people with speech changes, and participants completed about 90% of assigned recordings, suggesting home collection is feasible (NeurologyLive, Section 23). Your app builds on these same measures.
- Clinics commonly track ALS with a standard functional rating questionnaire. Resonance's Daily Function Check is **inspired by** that idea but uses original questions, so it is not a replacement for the clinical scale.
- Brain implants are starting to restore speech. In September 2026, Neuralink shared a video of Terry, a participant with bulbar-onset ALS, speaking through an implant in a synthetic voice modeled on his voice from before ALS. The devices are investigational and only available to trial volunteers. Resonance is the tool people can use **today**, with an iPhone, and its Voice Bank preserves the voice recordings that make a personal voice possible later.
- Apple's **Personal Voice** (iOS 17+) lets a user create a synthesized version of their voice by reading prompts (about 150, roughly 15 minutes). Third-party apps can use it with the user's permission (Section 14).

4. Product overview

The seven modules

Module	What the user does	What the app produces	In Monday MVP?
Voice Lab	3 short voice tasks, about 2 minutes, weekly	12 speech measures per session	Yes
Motor Lab	Finger tapping and a phone-steadiness hold, about 1.5 minutes, weekly	6 motor measures per session	Yes (spiral = stretch)
Daily Function Check	8 quick questions plus mood, about 30 seconds, daily or weekly	Function score 0 to 32	Yes
Insights Engine	Nothing; runs automatically	Baseline, change %, domain scores, Function Index, flags, weekly summary	Yes
AI Speech Change Score	Nothing; runs on Voice Lab audio	Probability-style score 0 to 100 from an on-device Core ML model	Stretch (Sunday if time)
Talk board	Speaks phrases by touch, head pointer or eye gaze	Spoken phrases in the user's voice	Yes (eye gaze = stretch)
Voice Bank	Records personal phrases; sets up Personal Voice	Stored clips; Personal Voice access	Yes
Care Report	Taps Export	PDF of trends to share with the care team	Yes

Users

Person	Example	What they need
Person with ALS	Dana, 58, diagnosed 6 months ago with bulbar-onset ALS; speech is starting to slow	Quick weekly checks, a way to keep talking, control over their data
Caregiver	Her son Marcus, 24, helps with appointments	To see trends, set up the Talk board, record and organize phrases
Care team	Dana's neurologist and speech-language pathologist	A clear one-page summary between visits

A week with Resonance

1. **Monday morning:** a reminder appears. Dana does the 2-minute Voice Lab and 90-second Motor Lab in her kitchen.
2. **Every day:** she answers the 30-second Function Check. On a tired day she rates fatigue 3 of 4.
3. **Anytime:** she uses the Talk board, saying "I love you" to Marcus in her own banked voice.
4. **Sunday:** the Today screen shows a weekly summary: speaking rate is down 6% from her baseline, finger tapping is stable.
5. **Before a clinic visit:** Marcus exports the Care Report PDF and emails it to her speech-language pathologist.
6. **When speech gets harder:** they switch the Talk board to head-pointer mode so Dana can talk without touching the screen.

5. Screens and navigation

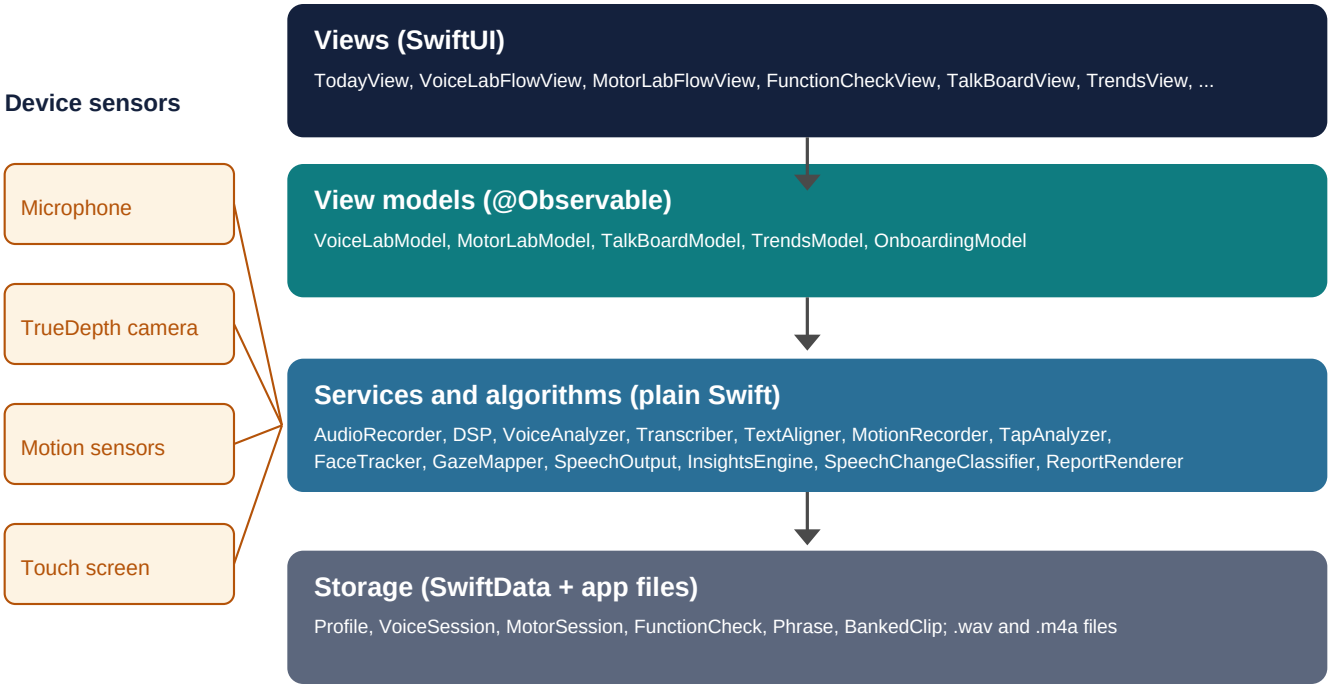
A **TabView** with five tabs. Use large type, high contrast, and tap targets of at least 60 points everywhere, because users may have weak hands.

Tab	Screen	Contents
Today	TodayView	Greeting; cards for due tasks (Voice Lab, Motor Lab, Function Check) with a checkmark when done; Function Index ring; weekly summary text; streak
Check-in	CheckInHubView	Buttons to start Voice Lab, Motor Lab, Function Check; last completed date for each
	VoiceLabFlowView	Intro -> quiet calibration (1 s) -> Task A held "ahh" -> Task B "pa-ta-ka" -> Task C reading passage -> results summary
	MotorLabFlowView	Intro -> tapping right hand -> tapping left hand -> steadiness hold -> results
	FunctionCheckView	8 questions, one per screen, big buttons 0 to 4, plus mood faces
Talk	TalkBoardView	Category bar; 2x3 grid of phrase tiles; input mode indicator; last 10 phrases; keyboard button; emergency tile
	KeyboardView	Large letter grid with word predictions; Speak button
	CalibrationView	5-dot calibration for head pointer or eye gaze
Trends	TrendsView	Function Index line chart; domain charts (Speech, Motor, Daily Function); per-metric detail with baseline line; AI Speech Change Score chart
	ReportPreviewView	Care Report preview with Export and Share buttons
Profile	ProfileView	Name, diagnosis date (optional), onset type (bulbar / limb / not sure), reminders, input mode, dwell time, voice settings
	VoiceBankView	Personal Voice status and setup guide; recorded phrase list with record, play, re-record, assign to tile

Tab	Screen	Contents
	DataPrivacyView	What is stored, export all data, delete all data, demo mode toggle
First launch	OnboardingFlow	Welcome -> who is using the app -> not-a-diagnosis consent -> permissions (microphone, speech recognition, motion, camera) -> schedule

6. Architecture

Four layers. Views only talk to view models; view models call services; services do the real work and write SwiftData models. Sensors feed services only. Algorithms are pure Swift, so they are easy to unit test.



No network. All processing and storage stay on the iPhone.

Key Apple frameworks

Framework	Used for
AVFoundation (AVAudioSession, AVAudioEngine, AVAudioFile, AVAudioPlayer)	Recording voice tasks as 44.1 kHz mono audio; playing banked clips
Speech (SFSpeechRecognizer)	On-device transcription with word timestamps for the reading passage
Accelerate (vDSP)	Fast signal math: RMS energy, autocorrelation, FFT
CoreMotion (CMMotionManager)	100 Hz device motion for the steadiness hold
ARKit (ARFaceTrackingConfiguration, ARFaceAnchor)	Head pose and eye blend shapes for hands-free pointing and blink selection
AVFAudio (AVSpeechSynthesizer)	Speaking text, including the user's Personal Voice when authorized
Charts (Swift Charts)	Trend charts
SwiftData	Local storage of sessions, check-ins and phrases

Framework	Used for
CoreML + Create ML	Optional on-device Speech Change Score model
UserNotifications	Weekly and daily reminders
SwiftUI ImageRenderer + ShareLink	Rendering and sharing the Care Report PDF

7. Xcode project setup

1. Xcode -> File -> New -> Project -> iOS -> **App**. Product name **Resonance**. Interface **SwiftUI**. Language **Swift**. Storage **SwiftData**. Check **Include Tests**.
2. Target -> General -> Minimum Deployments: **iOS 17.0**. Supported destinations: iPhone. Orientation: Portrait only.
3. Target -> Signing and Capabilities: select your Personal Team (free Apple ID). Plug in your iPhone, enable Developer Mode on it, and run once to confirm signing works.
4. Target -> Info: add the privacy keys below. The app crashes the moment it touches a sensor without its key.
5. Create the folder groups shown in the file tree. Add **Passage.txt** and **Phrases.json** to Resources (contents in Sections 9 and 13).
6. Create a **GitHub** repository now and commit after every task. The submission needs a link to your code.

Info.plist privacy keys

Key	Suggested text shown to the user
NSMicrophoneUsageDescription	Resonance records short voice tasks to measure speech changes and lets you record phrases in your own voice. Recordings stay on this iPhone.
NSSpeechRecognitionUsageDescription	Resonance transcribes your reading on this iPhone to measure speaking rate and clarity.
NSMotionUsageDescription	Resonance uses motion sensors to measure how steadily you can hold your phone.
NSCameraUsageDescription	Resonance uses the front camera to let you control the Talk board with your head or eyes. Video is never saved.

File tree

```
Resonance/
  App/          ResonanceApp.swift, RootTabView.swift, AppTheme.swift
  Models/       Profile.swift, VoiceSession.swift, MotorSession.swift, FunctionCheck.swift,
                Phrase.swift, BankedClip.swift, MetricDefinitions.swift
  Services/
    Audio/      AudioRecorder.swift, DSP.swift, PitchTracker.swift, VoiceAnalyzer.swift
    Speech/     Transcriber.swift, TextAligner.swift, SyllableCounter.swift
    Motor/      MotionRecorder.swift, TapAnalyzer.swift, SteadinessAnalyzer.swift
    Gaze/       FaceTracker.swift, GazeMapper.swift, DwellSelector.swift, BlinkDetector.swift
    Output/     SpeechOutput.swift, ClipPlayer.swift
    Insights/   InsightsEngine.swift, Stats.swift, SummaryWriter.swift
    ML/         SpeechChangeClassifier.swift          (stretch)
    Report/     ReportView.swift, ReportRenderer.swift
    Demo/       DemoDataSeeder.swift
    Reminders/  ReminderScheduler.swift
  Features/
    Onboarding/ Today/ CheckIn/ VoiceLab/ MotorLab/ FunctionCheck/
    Talk/ VoiceBank/ Trends/ Profile/
  Resources/    Passage.txt, Phrases.json, Assets.xcassets, SpeechChange.mlmodel (stretch)
Scripts/        TrainSpeechModel.swift (stretch: Create ML training, run on the Mac)
PLAN.pdf        this document
CLAUDE.md       standing instructions for Claude Code
ResonanceTests/ DSPTests.swift, PitchTrackerTests.swift, TextAlignerTests.swift,
                TapAnalyzerTests.swift, InsightsEngineTests.swift, GazeMapperTests.swift
```

Device requirements

Microphone, motion and TrueDepth face tracking only work on a real iPhone, not the Simulator. Use the Simulator for layout only. Face tracking needs an iPhone with Face ID; check `ARFaceTrackingConfiguration.isSupported` and fall back to touch.

8. Data model (SwiftData)

Store every computed metric as a named value in a dictionary so new measures can be added without changing the schema. Raw audio is saved as files; the model stores only the file name.

```
import SwiftData
import Foundation

@Model final class Profile {
    var name: String
    var role: String              // "person", "caregiver", "demo"
    var onsetType: String         // "bulbar", "limb", "unsure"
    var diagnosisDate: Date?
    var dwellSeconds: Double = 1.2
    var inputMode: String = "touch"   // "touch", "head", "eyes"
    var preferPersonalVoice: Bool = true
    var createdAt: Date = Date()
    init(name: String, role: String, onsetType: String) {
        self.name = name; self.role = role; self.onsetType = onsetType
    }
}

@Model final class VoiceSession {
    var date: Date
    var metrics: [String: Double]       // keys from MetricKey (Section 9)
    var vowelFile: String?              // file names in Documents/Audio
    var ddkFile: String?
    var readingFile: String?
    var transcript: String?
    var speechChangeScore: Double?      // 0-100, stretch
    var qualityFlags: [String] = []     // e.g. "too_noisy", "too_short"
    var isDemo: Bool = false
    init(date: Date, metrics: [String: Double]) { self.date = date; self.metrics = metrics }
}

@Model final class MotorSession {
    var date: Date
    var metrics: [String: Double]
    var qualityFlags: [String] = []
    var isDemo: Bool = false
    init(date: Date, metrics: [String: Double]) { self.date = date; self.metrics = metrics }
}

@Model final class FunctionCheck {
    var date: Date
    var answers: [Int]      // 8 answers, each 0...4 (4 = normal)
    var mood: Int           // 1...5
    var note: String?
    var isDemo: Bool = false
    var total: Int { answers.reduce(0, +) }     // 0...32
    init(date: Date, answers: [Int], mood: Int) {
        self.date = date; self.answers = answers; self.mood = mood
    }
}
```

```
@Model final class Phrase {
    var text: String
    var category: String    // "Needs", "Feelings", "People", "Replies", "Custom"
    var order: Int
    var clip: BankedClip?   // if set, play own-voice recording instead of TTS
    var useCount: Int = 0
    init(text: String, category: String, order: Int) {
        self.text = text; self.category = category; self.order = order
    }
}

@Model final class BankedClip {
    var fileName: String    // Documents/Bank/<uuid>.m4a
    var label: String
    var recordedAt: Date
    init(fileName: String, label: String) {
        self.fileName = fileName; self.label = label; self.recordedAt = Date()
    }
}
```

Metric definitions

Define every metric once in `MetricDefinitions.swift` with its key, display name, unit, domain, and whether higher is better. The Insights Engine and charts read from this list.

```
enum Domain: String { case speech, motor, daily }

struct MetricDef {
    let key: String; let name: String; let unit: String
    let domain: Domain; let higherIsBetter: Bool; let inIndex: Bool
    init(_ k: String, _ n: String, _ u: String, _ d: Domain, _ hib: Bool, _ idx: Bool) {
        key = k; name = n; unit = u; domain = d; higherIsBetter = hib; inIndex = idx
    }
}

// key, display name, unit, domain, higherIsBetter, inFunctionIndex
let metricDefs: [MetricDef] = [
  .init("mpt",             "Max phonation time",  "s",      .speech, true,  true),
  .init("f0Mean",          "Average pitch",       "Hz",     .speech, true,  false),
  .init("f0StdSemi",       "Pitch variation",     "st",     .speech, false, false),
  .init("jitter",          "Pitch steadiness",    "%",      .speech, false, false),
  .init("shimmer",         "Loudness steadiness", "%",      .speech, false, false),
  .init("ddkRate",         "Pa-ta-ka rate",       "syl/s",  .speech, true,  true),
  .init("ddkCV",           "Pa-ta-ka rhythm",     "CV",     .speech, false, false),
  .init("wpm",             "Speaking rate",       "wpm",    .speech, true,  true),
  .init("sylPerSec",       "Syllable rate",       "syl/s",  .speech, true,  false),
  .init("pauseRatio",      "Pause time",          "%",      .speech, false, false),
  .init("intelligibility", "Word clarity",        "%",      .speech, true,  true),
  .init("loudnessDb",      "Reading loudness",    "dBFS",   .speech, true,  false),
  .init("tapRateR",        "Tap speed (right)",   "taps/s", .motor,  true,  true),
  .init("tapRateL",        "Tap speed (left)",    "taps/s", .motor,  true,  true),
  .init("tapAccuracy",     "Tap accuracy",        "%",      .motor,  true,  true),
  .init("tapCV",           "Tap rhythm",          "CV",     .motor,  false, false),
  .init("steadyRMS",       "Hold steadiness",     "mg",     .motor,  false, true),
  .init("fatigueRatio",    "Hold fatigue",        "ratio",  .motor,  false, false),
  .init("tremorBand",      "Tremor-band power",   "ratio",  .motor,  false, false),
]
```

9. Module spec: Voice Lab (audio analysis)

Three tasks, about 2 minutes total, done once a week (more often is fine). Each task shows a big animated cue, a countdown, and a live level meter so the user knows the phone is hearing them.

Recording setup (AudioRecorder.swift)

- `AVAudioSession`: category `.playAndRecord`, mode `.measurement` (turns off automatic gain and noise processing, which would distort measurements), options `[.defaultToSpeaker]`.
- `AVAudioEngine` input tap, buffer 4096 frames, converted to **44,100 Hz mono Float32**. Keep samples in a `[Float]` array and also write a .wav file with `AVAudioFile` to Documents/Audio.
- Publish a live level (RMS in dBFS, updated about 20 times per second) for the on-screen meter.
- Ask the user to hold the phone about 30 cm (one foot) from their mouth, in the same quiet room each week. Show this in the intro screen.

Step 0: quiet calibration (1 second)

Record 1 second of silence. Noise floor = 10th percentile of frame dB. If the noise floor is above -45 dBFS, show "It's a bit noisy here. Try a quieter room." and add the quality flag `too_noisy` (the user can continue).

Framing (DSP.swift)

```
// 44,100 Hz. Frame = 25 ms (1102 samples), hop = 10 ms (441 samples).
func frameRMSdB(_ x: [Float], frame: Int = 1102, hop: Int = 441) -> [Float]
    // for each frame: rms = sqrt(mean(x^2)) via vDSP_rmsqv; dB = 20*log10(rms + 1e-9)
func movingAverage(_ x: [Float], width: Int) -> [Float]
func percentile(_ x: [Float], _ p: Float) -> Float
func voicedMask(_ db: [Float], noiseFloor: Float, margin: Float = 15) -> [Bool]
    // voiced if db > noiseFloor + margin; then close gaps shorter than 10 frames (100 ms)
```

Task A: held "ahh" (sustained vowel)

Prompt: "Take a deep breath, then say 'ahh' steadily for as long as you comfortably can." Max recording 30 s; stops automatically after 1.5 s of silence following voice.

Metric (key)	How to compute
Max phonation time (mpt)	Longest continuous voiced run in seconds (from voicedMask). Reject if under 1 s (flag too_short).
Average pitch (f0Mean)	PitchTracker on each voiced frame (method below); mean of valid F0 values in the middle 80% of the run.
Pitch variation (f0StdSemi)	Standard deviation of 12*log2(f0 / f0Mean) across frames, in semitones.
Pitch steadiness (jitter)	Frame-level approximation: mean(|T(i) - T(i-1)|) / mean(T) x 100, where T = 1/f0 per frame. Label it "approximate jitter".
Loudness steadiness (shimmer)	Frame-level approximation: mean(|A(i) - A(i-1)|) / mean(A) x 100, where A = peak absolute amplitude per frame.

PitchTracker (autocorrelation, simplified YIN)

```
// Input: one 40 ms frame (1764 samples), Hann window applied.
// Search lags for 75 Hz to 400 Hz: minLag = 44100/400 = 110, maxLag = 44100/75 = 588.
// 1. d(tau) = sum_j (x[j] - x[j+tau])^2              (difference function, use vDSP)
// 2. d'(tau) = d(tau) * tau / sum_{k=1..tau} d(k)     (cumulative mean normalized)
// 3. first tau in [minLag, maxLag] where d'(tau) < 0.15 and is a local minimum
//    (if none: frame is unvoiced, return nil)
// 4. parabolic interpolation around tau for sub-sample accuracy
// 5. f0 = 44100 / tau
// Reject frames whose f0 jumps more than 20% from the median of neighbors (octave errors).
```

Task B: "pa-ta-ka" repetition (DDK)

Prompt: "Say 'pa-ta-ka' over and over, as quickly and clearly as you can, until the timer ends." Record 8 seconds. Show a pulsing circle so the user knows when to start and stop.

```
// SyllablePeaks in VoiceAnalyzer.swift
// 1. env = frameRMSdB(x, frame: 441, hop: 441)          // 10 ms frames
// 2. env = movingAverage(env, width: 3)
// 3. ignore the first 0.5 s and any trailing silence
// 4. peaks = local maxima where env > noiseFloor + 10 dB,
//            prominence >= 6 dB over the lowest point since the previous peak,
//            at least 70 ms (7 frames) after the previous peak
// 5. ddkRate = (peaks.count - 1) / (time(lastPeak) - time(firstPeak))   // syllables per second
// 6. intervals = diff(peakTimes); ddkCV = std(intervals) / mean(intervals)
// Quality: if peaks.count < 8, flag "too_few_syllables" and do not save ddk metrics.
```

Task C: reading passage

Show this original passage in large text (store it in Passage.txt). Using an original passage avoids copyright issues and stays identical every week, which makes comparisons fair.

Passage.txt

On Saturday morning my brother and I walked to the farmers market by the lake. The air was cool, and the sky was a pale shade of blue. We bought peaches, fresh bread, and a jar of honey for our grandmother. A man played a quiet tune on his guitar while children chased a puppy around the fountain. Before we left, we shared a warm cinnamon roll and talked about our plans for the week.

Metric (key)	How to compute
Transcript	SFSpeechURLRecognitionRequest on the saved .wav; requiresOnDeviceRecognition = true (check supportsOnDeviceRecognition first); shouldReportPartialResults = false. Use result.bestTranscription.segments (each has substring, timestamp, duration).
Speaking rate (wpm)	recognized word count / (end of last segment - start of first segment) x 60.
Syllable rate (sylPerSec)	Syllables in the recognized words (SyllableCounter: count vowel groups, subtract silent final e, minimum 1 per word) / same duration.
Pause time (pauseRatio)	Sum of gaps longer than 250 ms between consecutive segments / total duration x 100.
Word clarity (intelligibility)	TextAligner: word-level Levenshtein alignment of the normalized transcript against the passage; matched words / passage words x 100. This is an automatic proxy for how understandable speech is, not a clinical score.
Reading loudness (loudnessDb)	Median frame dB of voiced frames during reading.

On-device transcription check

Check `SFSpeechRecognizer(locale: Locale(identifier: "en-US"))?.supportsOnDeviceRecognition` before Task C. If it is false, do not silently fall back to Apple's server recognition, which would send audio off the phone. Instead, ask the user once whether to allow it, and if they say no, skip the four transcript-based metrics and keep everything else.

TextAligner (word-level edit distance)

```
// normalize: lowercase, strip punctuation, split on whitespace
// dp[i][j] = min( dp[i-1][j] + 1,                    // deletion (passage word missed)
//                 dp[i][j-1] + 1,                    // insertion (extra word)
//                 dp[i-1][j-1] + (ref[i]==hyp[j] ? 0 : 1) )   // match or substitution
// backtrack to count exact matches; intelligibility = matches / ref.count * 100
// Also return the list of missed passage words for the results screen.
```

Results screen

- Show 4 headline numbers (speaking rate, pa-ta-ka rate, max phonation time, word clarity), each with an arrow and percent versus baseline once a baseline exists.
- Show the transcript with missed words in gray, so the person can see it worked.
- If any quality flag is set, show a calm note and offer "Redo this task".
- Save a VoiceSession with all metrics, file names and flags. Then call the Insights Engine to update trends.

Done when

- A 220 Hz test tone played into the microphone reports f0Mean within 3% (also covered by a unit test with a synthetic sine).
- Saying "pa-ta-ka" at a normal pace gives a ddkRate between about 4 and 7 syllables per second for a typical healthy teen voice.
- Reading the passage normally gives wpm in a plausible range (roughly 130 to 200) and intelligibility above 85% in a quiet room.
- All three recordings are saved as files and the session appears on the Trends tab.

10. Module spec: Motor Lab (motor assessment)

About 90 seconds. Tests hand speed, accuracy and steadiness, which ALS can affect when limbs are involved.

Test 1: alternating finger tapping (each hand, 10 seconds)

- Two circular targets, 80 pt diameter, centers 160 pt apart, in the middle of the screen. Instructions: "Use your index finger to tap the two circles one after the other, as fast as you can."
- Capture every touch with a `DragGesture(minimumDistance: 0)` onEnded handler: location and time from `CACurrentMediaTime()`.
- Run the right hand, then the left hand. Show a 3-2-1 countdown and a 10-second progress bar.

Metric	Formula
tapRateR / tapRateL	Valid taps / 10 s. A valid tap lands inside a target AND on the opposite target from the previous valid tap.
tapAccuracy	Taps inside either target / all taps x 100 (both hands combined).
tapCV	Standard deviation / mean of intervals between valid taps (both hands, averaged).

Test 2: steadiness hold (15 seconds)

- Instruction: "Hold your phone straight out in front of you with your arm extended, as still as you can." (Seated users may rest the elbow; record the posture choice in settings so it stays consistent.)
- `CMMotionManager.deviceMotionUpdateInterval = 1.0 / 100`. Record `userAcceleration` (gravity removed), x, y, z in g.
- Discard the first 2 seconds (settling).

```
// SteadinessAnalyzer
// mag[i] = sqrt(ax^2 + ay^2 + az^2) * 1000          // milli-g
// steadyRMS    = sqrt(mean(mag^2))                    // lower = steadier
// fatigueRatio = rms(last 4 s) / rms(first 4 s)       // > 1 means shakier as the arm tires
// tremorBand   = power in 3-12 Hz / total power, via vDSP FFT (1024-point, Hann window)
//                stored as an extra metric "tremorBand" (not in the index)
```

Test 3 (stretch): spiral trace

Draw an Archimedean spiral r = 8 + 6 x theta points for theta 0 to 6 pi. The user traces it with a finger. Metric: mean distance in points from each touch sample to the nearest point on the ideal spiral, and completion time. Add only if time allows.

Done when

- Tapping produces rates for both hands, accuracy and rhythm; resting the phone on a table during the hold gives a very low steadyRMS.
- A MotorSession is saved and appears on Trends. Unit tests for TapAnalyzer pass with scripted tap sequences.

11. Module spec: Daily Function Check (patient-reported data)

Eight questions about everyday function, one per screen, with five big answer buttons. 4 = normal, 0 = unable. These are original questions inspired by how ALS clinics track function; they are **not** the official clinical scale. Total = 0 to 32.

#	Question	4	2	0
1	How is your speech today?	Normal	Others sometimes ask me to repeat	I can't be understood
2	How is swallowing and saliva today?	Normal	Some coughing or drooling	I can't swallow safely
3	How are writing and small hand tasks (buttons, phone)?	Normal	Slow or clumsy	I can't do them
4	How are dressing and personal care?	Normal	I need some help	I need full help
5	How is walking?	Normal	I use a cane or walker	I can't walk
6	How are stairs?	Normal	Slow, need the rail	I can't do stairs
7	How is breathing when lying down or active?	Normal	Short of breath sometimes	I need breathing support
8	How is your energy (fatigue)?	Normal	Tired by midday	Too tired for most things

Answers 3 and 1 are in between (label them "Mostly normal" and "Very hard"). Then a mood question with five faces (1 to 5) and an optional note, which the Talk board's keyboard can fill in.

Safety rule

If question 7 (breathing) is answered 0 or 1, or drops by 2 or more points from the last check, show: "Breathing changes are important. Please contact your care team. If you are having trouble breathing right now, call 911." Never hide or delay this message.

12. Module spec: Insights Engine and AI Speech Change Score

The Insights Engine compares each person to **their own baseline**, separates real change from normal day-to-day noise, and produces scores, flags and a plain-English weekly summary. All code is pure Swift in Services/Insights.

Step 1: baseline

- Baseline for each metric = **median of the first 3 sessions** (or all sessions in the first 14 days if more). Median resists one bad recording.
- Baseline noise for each metric = standard deviation of those baseline sessions (minimum: 3% of the baseline value, so a lucky-consistent baseline does not over-trigger).
- Until 3 sessions exist, show "Building your baseline: 1 of 3 sessions" instead of scores.

Step 2: per-metric change

```
// direction d = +1 if higherIsBetter else -1
// changePct   = d * (current - baseline) / baseline * 100     // negative = worse
// ratio       = current / baseline   (higherIsBetter)
//             = baseline / current   (lower is better)        // 1.0 = same as baseline
// metricScore = min(max(ratio, 0), 1.0) * 100                  // 100 = at or better than baseline
```

Step 3: domain scores and Function Index (0 to 100)

Score	Built from	Weight in Function Index
Speech	Mean metricScore of index metrics in the speech domain: mpt, ddkRate, wpm, intelligibility	40%
Motor	Mean metricScore of tapRateR, tapRateL, tapAccuracy, steadyRMS	35%
Daily function	Latest FunctionCheck total / 32 x 100 (relative to its own baseline the same way)	25%

Function Index = weighted mean, capped at 100. The weights are design choices; explain them in your video and let the user view each domain separately. If a domain has no data yet, renormalize the remaining weights.

Step 4: is the change real? (trend detection)

```
// For each index metric, using the last 6 sessions (at least 4):
// slope = least-squares slope of value vs. days; weeklyChange = slope * 7
// projected4wk = d * slope * 28 / baseline * 100                 // % change over 4 weeks
// flag "noticeable change" when ALL are true:
//   1. the last 2 sessions are both worse than baseline by more than 2 x baselineNoise
//   2. projected4wk < -10 (worsening faster than 10% per 4 weeks)
//   3. no quality flags on those sessions
// flag "improved" symmetrically. Otherwise "stable".
```

This two-part rule (bigger than normal noise AND a consistent trend) is the statistical heart of the app. Judges with STEM backgrounds will respect that you thought about false alarms. Put a short explanation on the Trends tab: "Why we say 'stable'".

Step 5: weekly summary text (SummaryWriter.swift)

Template-based, no network. Example output: *"This week: speaking rate 142 wpm (down 6% from your baseline, within your normal range). Pa-ta-ka rate stable. Finger tapping stable. One noticeable change: max phonation time has decreased over the last 4 sessions. Consider mentioning this to your care team."* Always end flags with a care-team suggestion, never a conclusion about the disease.

AI Speech Change Score (stretch, Core ML)

This is the part most similar to Synapse Match's AI prediction. Build it only after the MVP works. No API key is needed: you train the model once on your Mac with Apple's free Create ML, and it runs on the phone.

Datasets (free to download)

Dataset	What is in it	Access	Use it for
VOC-ALS (recommended)	153 people (102 with ALS, 51 healthy controls); 8 recordings each: sustained vowels a, e, i, o, u and repeated pa, ta, ka. WAV, 8 kHz, 16-bit, plus a spreadsheet of clinical and acoustic data. Recorded in Italy.	Synapse, ID syn53009474. Free registered account; follow Synapse's terms of use and cite the 2024 Scientific Data paper.	Main model. Its tasks match your Voice Lab Tasks A and B exactly.
Mendeley "ALS Disease patient classification"	148 sustained-vowel recordings (/a/ and /i/) from people with ALS and healthy controls, recorded on smartphones at 44.1 kHz.	Direct download, CC BY 4.0. It appears to match the Minsk smartphone vowel set described by Vashkevich and colleagues; cite both.	Extra vowel data or a second test set
Minsk vowel set (GitHub)	Sustained /a/ from 15 people with bulbar ALS and 39 healthy speakers, smartphone recordings.	Public GitHub repository linked from the arXiv paper.	Extra test set
TORGO	Words and sentences from speakers with dysarthria from cerebral palsy or ALS, and control speakers.	Free for academic, non-profit use; must cite the TORGO papers.	Optional reading-speech model
Speech Accessibility Project	Large set including people with ALS	Requires an institution's signed data use agreement and a proposal; not realistic this weekend.	Future work

1 **Download:** create a free Synapse account, open dataset syn53009474 (VOC-ALS) and download the phonationA and rhythmPA/TA/KA folders plus the spreadsheet. Read the dataset's terms first, keep it out of your public GitHub repository (add it to .gitignore), and cite it in your README.

2. **Prepare:** use the spreadsheet to label each speaker ALS or healthy. Make two folders per model, `als/` and `healthy/`. Split by **speaker** (about 80% train, 20% test) so the model is tested on voices it has never heard. Claude Code can write this script for you.
3. **Match the audio:** VOC-ALS is 8 kHz. Before classifying, downsample the app's 44.1 kHz recording to 8 kHz with `AVAudioConverter` so training audio and app audio sound alike.
4. **Train:** in Create ML (Xcode -> Open Developer Tool -> Create ML) choose **Sound Classification**, or have Claude Code write `Scripts/TrainSpeechModel.swift` using `MLSoundClassifier` from the CreateML framework. Train one model on the held vowel and, if time allows, one on pa-ta-ka. Record test accuracy. Export `SpeechChange.mlmodel`.
5. **Use:** in SpeechChangeClassifier.swift, run the model with `SNClassifySoundRequest` (SoundAnalysis framework) on the Task A (and Task B) recording. Average the "als" confidence across windows to get a score from 0 to 100.
6. **Show it as a trend** with the label in Section 21. Never show it alone as a verdict. Put the model's test accuracy and dataset limits on the app's About screen and in your video.

> **Honesty note for judges**
>
> These datasets are small, recorded on different equipment and in different languages, and the healthy group differs from your users in many ways. Say so. The score is useful for tracking one person over time, not for deciding whether someone has ALS.

13. Module spec: Talk board (hands-free communication)

The everyday feature. It must be fast, forgiving and usable with no hand movement.

Layout

- Top: category bar (Needs, Feelings, People, Replies, Custom). Middle: **2 x 3 grid** of large tiles (at least 150 x 150 pt), each with an icon (SF Symbol) and text. Bottom: Keyboard tile, Repeat last, and an Emergency tile in red.
- A visible pointer (circle) in head or eye mode, with a progress ring that fills during dwell.
- The last 10 spoken phrases in a horizontal strip for quick repeats.

Default phrases (Phrases.json)

```
[
 {"category": "Needs", "items": [
    "I'm thirsty", "I'm hungry", "Please reposition me", "I need the bathroom",
    "I'm too hot", "I'm too cold", "Please call the nurse", "I need my medication"]},
 {"category": "Feelings", "items": [
    "I love you", "I'm in pain", "I'm tired", "I'm happy", "I'm frustrated",
    "Thank you for helping me"]},
 {"category": "People", "items": [
    "Where is Marcus?", "Please call my daughter", "Can we video call?",
    "I want to see my friends"]},
 {"category": "Replies", "items": [
    "Yes", "No", "Maybe", "Please wait", "I don't know", "Please say that again"]},
 {"category": "Custom", "items": []}
]
```

Input modes

Mode	How it works	Select by
Touch (default)	Normal taps. Ignore taps shorter than 80 ms to filter accidental touches.	Tap
Head pointer (MVP)	ARKit face anchor transform: yaw and pitch angles of the head move the pointer. Most robust hands-free option.	Dwell (default 1.2 s) or deliberate blink
Eye gaze (stretch)	ARKit eye blend shapes or lookAtPoint, mapped to the screen after calibration. More natural but jumpier.	Dwell or deliberate blink

FaceTracker and GazeMapper

```
// FaceTracker: ARSession with ARFaceTrackingConfiguration (guard isSupported).
// Delegate didUpdate anchors -> ARFaceAnchor (about 60 Hz). Publish a FaceSample:
//   yaw, pitch    from anchor.transform (convert rotation matrix to Euler angles)
//   gazeX, gazeY  from blendShapes:
//       gazeX = ((eyeLookOutRight + eyeLookInLeft) - (eyeLookInRight + eyeLookOutLeft)) / 2
//       gazeY = ((eyeLookUpLeft + eyeLookUpRight) - (eyeLookDownLeft + eyeLookDownRight)) / 2
//       (check the sign on device; front camera is mirrored)
//   blinkL, blinkR from eyeBlinkLeft / eyeBlinkRight (0...1)
// Do not show the camera feed or save frames. A small "camera active" dot is enough.

// GazeMapper: 5-point calibration (center, 4 corners at 15% inset). At each dot,
// collect 1.0 s of samples, use the median (u, v) = (yaw, pitch) or (gazeX, gazeY).
// Fit screen = A * [u, v, 1] with least squares, separately for x and y
// (3 unknowns each, solve the 3x3 normal equations with simd).
// Smoothing: p = 0.25 * raw + 0.75 * previous (exponential moving average).
// Clamp to screen bounds. Save calibration in Profile; offer "Recalibrate" anytime.
```

DwellSelector and BlinkDetector

```
// DwellSelector: track which tile contains the pointer. If the same tile for
// dwellSeconds (Profile.dwellSeconds, adjustable 0.6...3.0) -> select, then 1.0 s cooldown.
// Moving off the tile for more than 150 ms resets progress (small tolerance for jitter).

// BlinkDetector: deliberate blink = both blinkL and blinkR > 0.7
// for between 0.4 s and 1.2 s. Natural blinks are shorter and are ignored.
// Optional setting "Blink to select" (off by default).
```

Speech output priority (SpeechOutput.swift)

1. If the phrase has a **BankedClip**, play the recording in the user's own voice (AVAudioPlayer).
2. Else if Personal Voice is authorized and preferred, speak with the Personal Voice (Section 14).
3. Else speak with a system voice (AVSpeechSynthesizer, rate about 0.45, volume 1.0).

Keyboard

Large letter grid in alphabetical order (easier for pointer users than QWERTY), 7 columns, with Space, Delete, Clear and Speak. Show three word predictions using `UITextChecker().completions(forPartialWordRange:in:language:)`, boosted by words the user typed before. Typed sentences can be saved as Custom phrases with one tap.

Emergency tile

Plays a loud repeating alert sound and flashes the screen until a caregiver taps Stop. It does not send messages automatically (iOS requires the user to confirm messages), and its label must not imply it contacts emergency services.

Done when

- Every tile speaks correctly by touch; a tile with a banked clip plays the clip.
- Head-pointer mode can select any of the 6 tiles within 5 seconds after calibration, with no accidental selections in 30 seconds of looking around the center.
- Dwell time and input mode can be changed in Profile and take effect immediately.

14. Module spec: Voice Bank

Personal Voice (Apple, iOS 17+)

- The user creates a Personal Voice in **Settings -> Accessibility -> Personal Voice** by reading about 150 prompts (roughly 15 minutes), then leaves the iPhone charging and locked while it processes (can take hours).
- In the same settings screen, the user must turn on **Allow Apps to Request to Use**; otherwise requests are denied automatically.
- In the app: call `AVSpeechSynthesizer.requestPersonalVoiceAuthorization`. If the status is authorized, read `AVSpeechSynthesisVoice.speechVoices()` and pick voices whose `voiceTraits` contain `.isPersonalVoice`. Use it in an `AVSpeechUtterance`.
- Handle every status: on older devices it reports denied; on the Simulator, unsupported. iOS cannot deep-link to the Personal Voice settings, so show step-by-step instructions with screenshots instead.
- VoiceBankView shows a status badge (Not set up / Waiting for permission / Ready) and a "Test my voice" button.

Message banking (own-voice recordings)

- List of suggested phrases to record while speech is still strong: "I love you", names of family members, "Good morning", favorite sayings, a laugh, "That's hilarious".
- Record with AVAudioRecorder to .m4a (AAC, 44.1 kHz, mono) in Documents/Bank; play back; re-record; delete; assign to a Talk board tile.
- Trim silence at the start and end automatically using the noise floor from DSP.
- Prompt gently, once a week, to record one more phrase. Never pressure.

> **Why this matters**
>
> Personal voices and future voice-restoring technology work best with recordings made before speech changes. Neuralink's trial participant Terry speaks through a voice modeled on his pre-ALS voice. Voice banking early is something a family can do today.

15. Care Report, onboarding, demo mode and settings

Care Report (ReportView + ReportRenderer)

- One or two pages: name, date range, Function Index chart, a table of each index metric (baseline, latest, change %, status: stable / noticeable change / improved), Function Check answers trend, breathing alerts, and user notes.
- Render a SwiftUI ReportView to PDF with `ImageRenderer` (render into a CGContext PDF, US Letter 612 x 792 pt). Share with `ShareLink`.
- Footer on every page: "Generated by Resonance, a student-built monitoring app. Measurements are estimates from a phone and are not a diagnosis."

Onboarding

1. Welcome: what the app does in three short cards (Measure, Talk, Share).
2. Who is using it: person with ALS, caregiver, or exploring with demo data.
3. Consent screen: the not-a-diagnosis statement (Section 21), data stays on the device, can be deleted anytime. Button: "I understand".
4. Permissions one at a time, each explained before the system prompt appears.
5. Schedule: pick a weekly check-in day and time; schedule notifications with UNCalendarNotificationTrigger.
6. Start baseline: "Let's do your first Voice Lab" (can skip).

Demo mode (needed for the video)

DemoDataSeeder creates 12 weeks of realistic synthetic sessions, all marked `isDemo = true`, with a gentle decline in speech measures starting at week 6 and stable motor measures, plus noise. Show a "Demo data" banner on every screen while demo mode is on. Use it in your video to show what trends look like over months; say clearly that it is sample data.

```
// week w = 0...11, baseline wpm 165, ddk 6.0, mpt 16, intelligibility 95, tapRate 5.5
// speech decline after week 5: value *= (1 - 0.012 * (w - 5)); noise: * (1 + N(0, 0.03))
// motor: stable with noise N(0, 0.04); FunctionCheck speech answer drops 4 -> 3 at week 8
```

Settings and accessibility

- Text size follows Dynamic Type; test at the largest accessibility size.
- Every control has a VoiceOver label. Minimum tap target 60 pt; Talk tiles much larger.
- Data and Privacy: export all data as CSV (via ShareLink), delete all data (with confirmation), toggle demo mode.

16. Build tasks with acceptance criteria

Do these in order. Hours are estimates for a student who knows basic Swift; an AI assistant can speed up the coding, but always test each task on your iPhone before moving on. **MVP** tasks are required for Monday; **Stretch** tasks only if ahead.

T01	Create the project and app shell	Fri | ~1 h
Depends on	None	
Files	ResonanceApp.swift, RootTabView.swift, AppTheme.swift, Info.plist	
Steps	1 Create the project as in Section 7, with iOS 17 target and the four privacy keys. 2 Build RootTabView with five tabs (Today, Check-in, Talk, Trends, Profile) using placeholder views and SF Symbols. 3 Define AppTheme colors and fonts (large, high-contrast). 4 Commit to GitHub.	
Done when	• App runs on your iPhone and shows five tabs. • Repository has the first commit.	

T02	Data model and metric definitions	Fri | ~1 h
Depends on	T01	
Files	Models/*.swift	
Steps	1 Add the SwiftData models from Section 8 and register them in the modelContainer. 2 Add MetricDefinitions.swift. 3 Add a debug button that inserts and lists a fake VoiceSession.	
Done when	• Fake sessions persist after closing and reopening the app.	

T03	Audio recorder and DSP basics	Fri | ~2 h
Depends on	T01	
Files	Services/Audio/AudioRecorder.swift, DSP.swift; Tests/DSPTests.swift	
Steps	1 Implement AudioRecorder (Section 9): session setup, engine tap, mono Float32 samples, .wav writing, live level. 2 Implement frameRMSdB, movingAverage, percentile, voicedMask with vDSP. 3 Build a test screen with a record button and a live level meter. 4 Unit tests: a synthetic 1 s sine at amplitude 0.5 gives RMS about -9 dBFS; silence gives below -80 dBFS.	
Done when	• Live meter moves when you speak; .wav file plays back correctly. • DSP tests pass.	

T04	Pitch tracker and sustained-vowel analysis	Sat | ~2 h
Depends on	T03	
Files	PitchTracker.swift, VoiceAnalyzer.swift; Tests/PitchTrackerTests.swift	
Steps	1 Implement the simplified YIN method in Section 9. 2 Implement mpt, f0Mean, f0StdSemi, jitter, shimmer. 3 Tests: synthetic sines at 110, 220 and 330 Hz return f0 within 2%; a 3 s tone gives mpt about 3 s.	
Done when	• Tests pass; a real held "ahh" gives believable numbers.	

T05	Syllable rate (pa-ta-ka) analysis	Sat | ~1.5 h
Depends on	T03	
Files	VoiceAnalyzer.swift; Tests/DSPTests.swift	
Steps	1 Implement the SyllablePeaks method and ddkRate, ddkCV. 2 Test: a synthetic signal of 30 ms tone bursts every 160 ms gives about 6.25 syllables per second.	
Done when	• Test passes; real recordings give 4 to 7 syl/s at a normal effort.	

T06	Transcription and reading-passage analysis	Sat | ~2 h
Depends on	T03	
Files	Transcriber.swift, TextAligner.swift, SyllableCounter.swift; Tests/TextAlignerTests.swift	
Steps	1 Request speech recognition permission; check on-device support. 2 Transcribe a .wav file with segments and timestamps. 3 Implement TextAligner and SyllableCounter; compute wpm, sylPerSec, pauseRatio, intelligibility, loudnessDb. 4 Tests: identical text -> 100%; one word dropped from 10 -> 90%; syllables of "honey" = 2, "walked" = 1.	
Done when	• Reading the passage produces all five metrics and a transcript.	

T07	Voice Lab flow UI	Sat | ~2 h
Depends on	T04, T05, T06	
Files	Features/VoiceLab/*	
Steps	1 Build intro, calibration, Task A, Task B, Task C and results screens (Section 9). 2 Big cues, countdowns, live level meter, auto-stop rules, redo buttons. 3 Save a VoiceSession with metrics, files and quality flags.	
Done when	• A full Voice Lab run takes under 3 minutes and saves a session.	

T08	Motor Lab: tapping and steadiness	Sat | ~2 h
Depends on	T02	
Files	MotionRecorder.swift, TapAnalyzer.swift, SteadinessAnalyzer.swift, Features/MotorLab/*; Tests/TapAnalyzerTests.swift	
Steps	1 Build the tapping screen and capture touches with timestamps. 2 Build the steadiness hold with CoreMotion at 100 Hz. 3 Compute the six motor metrics; save a MotorSession; results screen. 4 Tests: scripted alternating taps at 5 per second -> tapRate 5.0, accuracy 100%.	
Done when	• A full Motor Lab run saves a session with all metrics.	

T09	Daily Function Check	Sat | ~1 h
Depends on	T02	
Files	Features/FunctionCheck/*	
Steps	1 Eight one-question screens with big 0 to 4 buttons, mood faces, optional note. 2 Breathing safety message rule (Section 11). 3 Save a FunctionCheck.	
Done when	• Check-in takes under 45 seconds; breathing message appears when it should.	

T10	Insights Engine	Sat | ~2 h
Depends on	T02	
Files	InsightsEngine.swift, Stats.swift, SummaryWriter.swift; Tests/InsightsEngineTests.swift	
Steps	1 Implement baseline, noise, change %, metric and domain scores, Function Index, trend flags and summary text (Section 12). 2 Tests: flat data -> "stable"; a 2% weekly decline over 6 sessions -> "noticeable change"; one bad session -> no flag.	
Done when	• Tests pass; summary text reads naturally.	

T11	Trends tab and Today tab	Sat/Sun | ~2 h
Depends on	T10	
Files	Features/Trends/*, Features/Today/*	
Steps	1 Swift Charts line chart of Function Index; domain charts; metric detail with a dashed baseline line and shaded normal-noise band. 2 Today: due-task cards, Function Index ring, weekly summary.	
Done when	• Charts render with demo data and real sessions.	

T12	Demo data seeder	Sun | ~0.5 h
Depends on	T10	
Files	DemoDataSeeder.swift, Profile/DataPrivacyView.swift	
Steps	1 Generate 12 weeks of data as in Section 15; demo banner; toggle and clear.	
Done when	• Turning on demo mode fills Trends with a realistic 12-week story; turning it off removes it.	

T13	Talk board with touch and speech output	Sun | ~2 h
Depends on	T02	
Files	Features/Talk/*, SpeechOutput.swift, ClipPlayer.swift, Phrases.json	
Steps	1 Load default phrases into SwiftData on first launch. 2 Category bar, 2x3 tile grid with paging, recent strip, emergency tile. 3 SpeechOutput priority: banked clip, Personal Voice, system voice.	
Done when	• Every tile speaks; recent strip updates; emergency alert starts and stops.	

T14	Head-pointer control and dwell selection	Sun | ~2.5 h
Depends on	T13	
Files	FaceTracker.swift, GazeMapper.swift, DwellSelector.swift, CalibrationView.swift; Tests/GazeMapperTests.swift	
Steps	1 ARKit face tracking with yaw and pitch; 5-point calibration; least-squares mapping; smoothing. 2 Pointer overlay with dwell progress ring; adjustable dwell time. 3 Test: GazeMapper recovers a known linear mapping from 5 synthetic points within 1 pt.	
Done when	• Meets the Talk board "Done when" criteria in Section 13 for head-pointer mode.	

T15	Keyboard with word prediction	Sun | ~1 h
Depends on	T13	
Files	Features/Talk/KeyboardView.swift	
Steps	1 Alphabetical 7-column grid, predictions from UITextChecker, Speak, Save as phrase. 2 Works with touch and head pointer.	
Done when	• You can type and speak "see you tomorrow" using only the head pointer.	

T16	Voice Bank: Personal Voice and message banking	Sun | ~1.5 h
Depends on	T13	
Files	Features/VoiceBank/*	
Steps	1 Personal Voice authorization flow, status badge, setup instructions, Test my voice. 2 Record, trim, play, re-record, delete and assign clips to tiles.	
Done when	• A recorded phrase plays from its tile; Personal Voice works if set up on your phone (or shows the right status).	

T17	Care Report PDF	Sun | ~1 h
Depends on	T11	
Files	Services/Report/*, Features/Trends/ReportPreviewView.swift	
Steps	1 Build ReportView; render to PDF; preview; ShareLink.	
Done when	• Exported PDF opens in Files and looks clean on one or two pages.	

T18	Onboarding, reminders, privacy screens	Sun | ~1 h
Depends on	T01	
Files	Features/Onboarding/*, ReminderScheduler.swift, Profile/*	
Steps	1 Onboarding flow from Section 15; consent text from Section 21; notification scheduling; delete-all and CSV export.	
Done when	• Fresh install walks through onboarding once; reminders fire at the chosen time.	

T19	Polish, accessibility pass and device testing	Sun | ~1.5 h
Depends on	All MVP tasks	
Files	Everything	
Steps	1 Largest Dynamic Type check, VoiceOver labels, dark mode, empty states, error messages. 2 Run the full manual test checklist in Section 18.	
Done when	• No crashes during the full checklist; every screen readable at large text.	

T20	Stretch: eye-gaze mode and blink selection	Stretch | ~2 h
Depends on	T14	
Files	FaceTracker.swift, BlinkDetector.swift	
Steps	1 Gaze from blend shapes with calibration; deliberate-blink selection.	
Done when	• Can select a tile by eye alone after calibration in good light.	

T21	Stretch: AI Speech Change Score	Stretch | ~3+ h
Depends on	T06	
Files	Services/ML/*, Resources/SpeechChange.mlmodel	
Steps	1 Download VOC-ALS from Synapse; label and split by speaker; downsample app audio to 8 kHz; train with Create ML or MLSoundClassifier; integrate with SoundAnalysis; chart the score.	
Done when	• Model test accuracy recorded honestly; score appears on Trends with the required label.	

T22	Stretch: spiral trace	Stretch | ~1 h
Depends on	T08	
Files	Features/MotorLab/SpiralView.swift	
Steps	1 Draw the spiral, capture touches, compute mean deviation and time.	
Done when	• Metric saved and charted.	

T23 README and repository cleanup	Sun night | ~0.5 h
Depends on	All
Files	README.md
Steps	1 What it does, screenshots, how to build, frameworks used, how each metric works, limitations, AI-use disclosure, dataset credits.
Done when	• A judge can understand the project from the README alone.

T24 Video, submission	Sun night / Mon early | ~3 h
Depends on	All
Files	Video file; submission form
Steps	1 Film and edit the demo using the script in Section 19; upload; complete the submission checklist in Section 20.
Done when	• Submitted before Monday 9:00 a.m. Pacific, ideally Sunday night.

17. Four-day schedule

When	Tasks	Goal by the end
Fri evening (about 4 h)	T01, T02, T03	App shell runs on your phone; recording and DSP tested
Sat morning (about 5 h)	T04, T05, T06	All voice metrics computed and tested
Sat afternoon/evening (about 6 h)	T07, T08, T09, T10	Voice Lab, Motor Lab, Function Check and Insights working end to end
Sun morning (about 5 h)	T11, T12, T13, T14	Trends with demo data; Talk board with head pointer
Sun afternoon (about 4 h)	T15, T16, T17, T18, T19	All MVP features done and polished
Sun evening (about 4 h)	T23, T24	README, video filmed and edited, submission completed
Mon before 7 a.m.	Buffer only	Fix a problem if something went wrong; deadline is 9:00 a.m. Pacific

If you fall behind, cut in this order

1. All Stretch tasks (T20 to T22). The app is strong without them; mention them as "next steps" in the video.
2. Keyboard prediction (keep a plain keyboard).
3. Care Report PDF (show the Trends screen instead).
4. Steadiness hold (keep tapping).
5. **Never cut:** Voice Lab, Insights Engine, Trends with demo data, Talk board with touch and head pointer, banked clips. These are the story.

> **Reality check**
>
> This is an ambitious plan for one weekend. Using an AI coding assistant will help a lot, but testing on a real phone takes time. If by Saturday night the Voice Lab and Insights are not working, switch to the "cut" list immediately rather than staying up all night.

18. Testing plan

Unit tests (XCTest, ResonanceTests target)

Test file	What it checks
DSPTests	RMS of known sines; silence floor; moving average; voicedMask closes short gaps; percentile
PitchTrackerTests	Synthetic sines at 110, 220, 330 Hz within 2%; noise returns nil; octave-error rejection
TextAlignerTests	Perfect match 100%; deletions, insertions, substitutions counted correctly; punctuation ignored
TapAnalyzerTests	Scripted taps: rate, accuracy, CV; same-target double taps excluded
InsightsEngineTests	Baseline median; stable vs noticeable change vs improved; one-off bad session ignored; weights renormalize
GazeMapperTests	Least-squares fit recovers a known mapping; smoothing reduces jitter

Generate synthetic signals in the tests (sine waves, tone bursts), so no audio files are needed.

Manual test checklist (on your iPhone)

- Fresh install: onboarding appears once; every permission prompt shows your explanation text.
- Voice Lab in a quiet room and a noisy room (noisy shows the warning).
- Hold "ahh" for 5 s, then 15 s: mpt roughly matches a stopwatch.
- Read the passage slowly, then quickly: wpm changes in the right direction.
- Motor Lab: tapping slowly vs quickly; phone resting on a table vs held in a shaky hand.
- Breathing answer 0 shows the safety message.
- Demo mode on and off; Trends charts update.
- Talk board: all tiles by touch; head-pointer calibration and selection; recorded clip plays; Personal Voice status correct.
- Export the Care Report and open it.
- Airplane mode: everything still works (proves on-device processing).
- Largest text size and VoiceOver on: screens still usable.

Real-user feedback (strongly recommended)

Before Sunday night, show the app to at least two people: ideally someone affected by ALS, a speech-language pathologist, or someone from your local ALS Association chapter. If that is not possible this weekend, test with a grandparent or neighbor for usability, and plan the ALS-community pilot as a next step in your video. Write down one quote and one change you made because of feedback.

19. Demo video script (about 3 minutes)

Keep it under 3 minutes; some districts penalize longer videos. Record the phone screen with iOS Screen Recording and film your face or hands for transitions.

Time	On screen	What you say (adapt in your own words)
0:00-0:20	A family photo or a simple title card; then the Neuralink headline	"ALS slowly takes away a person's movement and voice. This fall, a man with ALS told his wife 'I love you' through a brain implant. But implants are experimental and rare. I wanted to build something people with ALS can use today, with just an iPhone."

Time	On screen	What you say (adapt in your own words)
0:20-0:40	Problem graphic	"Speech and hand changes are usually measured at clinic visits that can be months apart. And when speech fades, people lose their connection to the people around them."
0:40-1:20	Voice Lab live: ahh, pa-ta-ka, reading; results screen	"Resonance's weekly Voice Lab takes two minutes. It measures 12 features of speech, like speaking rate and how fast you can repeat syllables, measures researchers have studied for tracking ALS."
1:20-1:45	Motor Lab tapping and hold; Function Check	"The Motor Lab tests finger speed and steadiness, and a 30-second check-in records daily function."
1:45-2:10	Trends with demo data (banner visible); flag; Care Report export	"Resonance compares each person to their own baseline and only flags a change when it's bigger than normal day-to-day variation. Then it creates a report for the care team. This is demo data showing 12 weeks."
2:10-2:40	Talk board controlled by head movement; banked clip says "I love you"	"And when speaking gets hard, the Talk board lets you talk hands-free, by moving your head, in your own recorded voice."
2:40-3:00	Airplane mode icon; you on camera	"Everything stays on the phone. Resonance doesn't diagnose; it helps people and their care teams see change sooner and keeps them talking. Next, I want to test it with the ALS community."

20. Submission checklist

- Registered at congressionalappchallenge.us under California's 10th District (team members: at least half live or attend school in the district).
- App runs on a real iPhone. Final commit pushed to a **public GitHub repository** with the README (T23).
- Demo video uploaded (YouTube unlisted or Vimeo) and the link tested in a private browser window.
- Submission form fields completed: app name, description, platform (iOS), languages (Swift), frameworks, the problem it solves, and the AI-use disclosure.
- Submitted before **Monday, October 26, 2026 at 12:00 p.m. Eastern (9:00 a.m. Pacific)**. Submissions cannot be changed after the deadline.
- Screenshot your confirmation. Watch for the exit questionnaire email afterward.

AI-use disclosure template

AI tools are allowed in the Congressional App Challenge as long as all use is fully disclosed. Edit this honestly to match what you actually did:

> **Disclosure (edit before submitting)**
>
> "I used Claude, an AI assistant, to help research past winners, plan the app's features and architecture, and generate parts of the Swift code, which I reviewed, tested on my iPhone, and modified. I designed the app's purpose, chose the measurements, decided how the insights and safety rules work, tested every feature, and created the video. [List any specific files or functions that were mostly AI-written.] The optional speech model was trained by me in Create ML using the VOC-ALS research dataset (Synapse), which I cite in my README."

21. Risks, ethics and privacy

Required wording

Place	Exact wording to use
Onboarding consent	Resonance helps you track changes in speech and movement and communicate more easily. It does not diagnose ALS or any other condition, and it does not replace your care team. Phone measurements are estimates and can be affected by noise, posture and how you feel that day.
Noticeable change flag	This measure has changed more than your usual day-to-day variation. It may be worth mentioning to your care team.
AI Speech Change Score	How much this recording resembles voice recordings from people with ALS in a research dataset. It is not a diagnosis and is most useful as a trend over time.
Emergency tile	Plays a loud alert to get attention nearby. It does not contact emergency services.

Risks and how the plan handles them

Risk	Mitigation
False alarms cause worry	Two-part rule (beyond normal noise AND consistent trend); calm wording; care-team framing
Recording conditions vary	Noise check, fixed distance instructions, quality flags, redo buttons, medians
Emotional weight of tracking decline	Show improvements and stability too; let users hide trends and use only the Talk board; never use alarming colors for normal results
Privacy of health data and voice	No network, local storage, camera frames never saved, delete-all button, export only when the user chooses
Overclaiming AI accuracy	Report the model's test accuracy and dataset limits in the app and video
Not tested with people who have ALS	Say so honestly; present a pilot with the ALS community as the next step
Weekend time pressure	MVP vs stretch split; cut list in Section 17

22. Prompts to give an AI coding assistant

With Claude Code, CLAUDE.md already loads the rules, so you can start with the per-task prompt. In any other setup, give the assistant this whole PDF and start with the starting prompt.

Starting prompt (once per session)

```
You are helping me build "Resonance", an iOS 17+ SwiftUI app, in Xcode. The attached plan
is the complete spec. Follow Section 1 "Rules for an AI coding assistant" exactly:
SwiftUI + SwiftData + @Observable, Apple frameworks only, no network, no third-party
packages, algorithms as plain testable Swift, never present outputs as a diagnosis.
Today we will do task <TXX> from Section 16. Read the module spec it references first.
Before writing code, list the files you will create or change. Then write complete,
compilable files (not fragments). Then give me the steps to test it on my iPhone and
the unit tests from Section 18 that apply.
```

Per-task prompt

```
Implement task <TXX>: <title>.
Spec: Section <N> of the plan (pasted below).
Files: <files from the task card>.
Done when: <paste the "Done when" list>.
Constraints: iOS 17, Swift, no packages, keep functions small, add comments explaining
the math. If something in the spec is ambiguous, choose the simplest option and tell me.
```

Debugging prompt

```
Task <TXX> builds but <describe exactly what happens on the phone>. Here is the Xcode
console output and the relevant file. Explain the cause in two sentences, then give the
smallest fix. Do not rewrite unrelated code.
```

Review prompt (run before submitting)

```
Review the whole project against Sections 9-15 and 21 of the plan. List: (1) any feature
in the MVP that is missing or broken, (2) any wording that could read as a diagnosis,
(3) any code that sends data off the device, (4) crash risks such as force unwraps on
sensor data. Order by importance.
```

23. Sources

Source	Link
Rep. DeSaulnier: 2025 CA-10 winner, Synapse Match	https://desaulnier.house.gov/media-center/press-releases/congressman-desaulnier-announces-2025-congressional-app-challenge-0
Rep. DeSaulnier: 2024 CA-10 winner, NeuroChess	https://desaulnier.house.gov/media-center/press-releases/congressman-desaulnier-announces-2024-congressional-app-challenge-0
Rep. DeSaulnier: 2023 CA-10 winner, CogniPro	https://desaulnier.house.gov/media-center/press-releases/congressman-desaulnier-announces-2023-congressional-app-challenge-0
Contra Costa News: 2022 winner, TrueDetect	https://contracosta.news/?p=3664
2026 deadline (Rep. McClain Delaney's office)	https://mcclaindelaney.house.gov/services/congressional-app-challenge
AI-use disclosure rule (Rep. Max Miller's office)	https://maxmiller.house.gov/posts/congressional-app-challenge
NeurologyLive: digital speech biomarkers in ALS	https://www.neurologylive.com/view/digital-speech-biomarkers-show-promise-recording-speech-function-dysarthria-als
MobiHealthNews: Neuralink implant helps man with ALS speak	https://www.mobihealthnews.com/news/neuralink-brain-implant-helps-man-als-speak-using-his-thoughts
iPhone in Canada: Neuralink ALS participant Terry	https://www.iphoneincanada.ca/2026/09/21/neuralink-als-patient-speaks-thoughts/
Ben Dodson: using Personal Voice in an iOS app	https://bendodson.com/weblog/2024/04/03/using-your-personal-voice-in-an-ios-app/
TORGO database, University of Toronto	https://www.cs.toronto.edu/~complingweb/data/TORGO/torgo.html
VOC-ALS paper (Scientific Data, 2024)	https://pmc.ncbi.nlm.nih.gov/articles/PMC11271596
VOC-ALS dataset on Synapse	https://doi.org/10.7303/syn53009474
Mendeley: ALS Disease patient classification (CC BY 4.0)	https://data.mendeley.com/datasets/fbhc38zzm9

Source	Link
Vashkevich et al.: bulbar ALS detection from voice (arXiv)	https://arxiv.org/pdf/2003.10806
Minsk vowel data on GitHub	https://github.com/Mak-Sim/Troparion/tree/master/SPA2019
Speech Accessibility Project: research access	https://speechaccessibilityproject.beckman.illinois.edu/conduct-research-through-the-project

Apple framework names and APIs in this plan reflect iOS 17 and later. Check Apple's developer documentation in Xcode (Help -> Developer Documentation) if an API behaves differently on your iOS version.